画像処理実験 第 6 回

学生番号: 09B24490
加藤弘都 (KATO, HIROTO)

2026 年 10 月 20 日

1　課題 5

1.1　[5-2]

1.1.1　条件 (i) – (iii) の意味の考察

類似度の表とは，表の片方の軸をある画像の特徴点とし，もう片方の軸も 2 つめの画像の特徴点とする．表の類似度の値はそれぞれの特徴点周囲の少領域の画素値の差の二乗和で類似度を示しており，値が小さいほど類似していることを意味する．以下の条件を満たすときの特徴点を探す．

- (i) 表の第 $[i, j]$ 要素の値が小さい対 (i, j) を選ぶ.
- (ii) 各 i は 1 つの j と対応する，
- (iii) 各 j は 1 つの i と対応する．

(i) は，類似度が小さいほどある特徴点同士が類似していることを利用している．

(ii)，(iii) では，ある特徴点に対応する別の画像の特徴点が複数存在すると，その点は別の画像の複数の離れた箇所と対応することになってしまう．このことを防ぐため，各特徴点には一つの特徴点が存在する条件が a 必要になる．

1.1.2　（方法 1）の問題点

（方法 1）の SSD の和は 939 であり，他の方法より大きくなっていることがわかる．方法 1 は i の値から順に最小の j を見つけている．このとき，見つけたときの j 列の値は無視されてしまう．すると，途中から無視される j の値がその列で選択されている j の値よりも小さいことがあるのだと考える．

つまり，問題点は先に見つけた j の値はその列の最小の j の値ではない可能性があるのに，それ以降ではその j 列を無視されてしまうということである．

1.1.3　（方法 2）の問題点

ある時点の最小の値と同じ行と列にある値は無視される．このことから，問題点は次以降の最小の値が無視される行と列に存在するとき，次の最小の値は大きくなり，結果として SSD も増加すると考える．

1.1.4 [6-2]

1.1.5 方法２の実装

方法２は以下のようになる．行列の最小の場所を探す．ji には座標ではなく，配列に登録されている値を前から順に 0,1,2,... のように数えた値が入る．そのため np.unravel_index 関数で ji を座標の形 (sx,sy) にする．最後に w に値を代入し，最小の値の列と行を除去する．

```
 1: def matchMethod2(mt,x1,x2):
 2:   N1=len(x1)
 3:   N2=len(x2)
 4:   N=min(N1,N2)
 5:   w=np.empty((N,4),'i')
 6: # mtorg=mt.copy() # 下記ループで mt が破壊されるため，必要であればコピーを保存しておく
 7:
 8:   for i in range(N):
 9:     ji=np.argmin(mt) # 行中の最小値の場所
10:     sx,sy=np.unravel_index(ji,mt.shape)
11:
12:     # 出力は対 (i,ji) ではなく，
13:     # それらに対応する座標の組 (x,y,u,v) とする
14:     w[i]=[ x1[sx ,0],x1[sx ,1],
15:            x2[sy,0],x2[sy,1] ]
16:     mt[:,sy]=np.inf      # 縦消去
17:     mt[sx,:]=np.inf      # 横消去
18:
19:   return w
```

また，課題５の detectFeaturePoints 関数を使ってそれぞれの画像の特徴点を配列 x1，x2 に格納し，getW 関数を呼び出す．これにより作られた W の値から対応を取ると図１のようになる．



図１　方法１（左）と方法２（右）

図１を見ると，対応を示す線が平行のものが多い場合は対応しているので，多くの点がうまく対応 i してると考える．しかし，いくつかの対応は片方の写真にそもそもないがため，対応がうまくできてなかったりしていると考える．

1.2 [6-3]

課題 6-2 の W の結果の中から正しい対応が取れた，４つの離れた点を以下のように選択する．

```
1:    w=np.array(
2:       [[119,194,367,209],
3:        [458,470,715,486],
4:        [137,456,388,463],
5:        [10,564,272,560]])
```

このとき，課題 3 の射影変換行列の計算する calcHomography 関数などから合成画像を生成すると図 2 のようになる．



図 2　[6-2] で求めた W のうち対応する 4 点（左）とそれを用いた合成画像（右）

図 2 を見ると,4 つの対応する点付近ではブレが少なく，正確に合成できていることがわかる．一部の家はぶれているので，正しい点を増やすことでより正確な画像になると考えた．

1.3　[6-4]

1.3.1　手法 3 と手法 1,2 の比較

表 1 を見ると，合計 SSD は方法 1 より小さくなっているが方法 2 と変わっていない．

表 1　合計 SSD の比較

	方法 1	方法 2	方法 3
合計 SSD	939	780	780

1.3.2　NAIVE と FAST の実行時間を比較

100 個の特徴点を求めるとき，calcSSDtable 関数の NAIVE と FAST の実行時間について表 2 で比較した．

表 2　変更前後の時間の変化

	FAST	NAIVE
類似値表の作成にかかる時間 (100 こ)	0.005	0.038
類似値表の作成にかかる時間 (10000 こ)	0.176	1.469

表 2 を見ると，FAST の方が早いことがわかる．これは，NAIVE では二重ループを使用しているため，処理が増えたのではないかと考えた．また，FAST 配列を X*X のようにまとめて計算していることから，処理が早くなったと考える．

2　感想

自動で正しい点をたくさん選択できたらもっといいのになーと感じた.

参考文献